

Dharma Satriadi (25120300030)

Soal 1 — innerHTML vs textContent

Dalam pengembangan aplikasi berbasis web, terdapat dua properti utama yang digunakan untuk memanipulasi isi suatu elemen HTML melalui Document Object Model (DOM), yaitu innerHTML dan textContent. Properti innerHTML berfungsi untuk membaca maupun menuliskan isi suatu elemen dalam bentuk markup HTML, sehingga string yang dimasukkan ke dalamnya akan diinterpretasikan dan dirender sebagai kode HTML oleh peramban (browser). Sebaliknya, properti textContent memperlakukan seluruh isi yang dimasukkan sebagai teks murni, tanpa melakukan proses penguraian (parsing) terhadap tag maupun atribut HTML di dalamnya.

Perbedaan mendasar tersebut membawa implikasi langsung pada aspek keamanan aplikasi. Penggunaan innerHTML untuk menampilkan data yang berasal dari input pengguna berpotensi menimbulkan celah keamanan berupa Cross-Site Scripting (XSS), sebagaimana diuraikan pada Soal 2, mengingat tag seperti <script> akan dieksekusi sebagai kode aktif apabila disisipkan melalui properti tersebut. Hal ini menyebabkan innerHTML sebaiknya hanya digunakan pada kondisi tertentu, yaitu ketika markup HTML memang perlu disisipkan dan sumber data telah dipastikan aman atau telah melalui proses penyaringan (sanitasi).

Sejalan dengan hal tersebut, penggunaan textContent direkomendasikan sebagai pilihan utama pada saat menampilkan data yang bersumber dari input pengguna maupun sumber eksternal yang belum terverifikasi keamanannya. Sebagai ilustrasi, penggunaan innerHTML dapat diamati pada contoh berikut:

```
elemen.innerHTML = "<strong>" + namaPengguna + "</strong>";  
// tag <strong> akan dirender sebagai elemen HTML aktif
```

Sementara itu, penerapan textContent yang lebih aman untuk kasus serupa adalah sebagai berikut:

```
elemen.textContent = namaPengguna;  
// seluruh isi namaPengguna diperlakukan sebagai teks, bukan markup
```

Berdasarkan uraian tersebut, dapat disimpulkan bahwa pemilihan antara innerHTML dan textContent perlu disesuaikan dengan kebutuhan penyisipan markup serta tingkat kepercayaan terhadap sumber data yang akan ditampilkan.

Soal 2 — Skenario Cross-Site Scripting (XSS)

Kerentanan yang Dihadapi

Salah satu skenario nyata yang menggambarkan risiko Cross-Site Scripting (XSS) adalah fitur kolom komentar pada suatu platform blog atau media sosial. Pada skenario tersebut, sistem menampilkan komentar yang dimasukkan pengguna secara langsung ke halaman menggunakan innerHTML tanpa melalui proses validasi maupun penyaringan (sanitasi) terlebih dahulu. Kondisi tersebut menyebabkan setiap tag maupun atribut event berbahaya yang disertakan pengguna turut dirender oleh peramban milik pengguna lain yang membuka halaman yang sama.

Proses Eksploitasi

Pelaku dapat memanfaatkan celah tersebut dengan memasukkan komentar yang berisi tag gambar () dengan atribut src yang sengaja dibuat tidak valid, sehingga atribut onerror miliknya akan dieksekusi oleh peramban. Pendekatan ini perlu ditegaskan, mengingat tag <script> yang disisipkan melalui innerHTML tidak akan dieksekusi oleh peramban modern, sehingga celah XSS pada praktiknya lebih sering dieksploitasi melalui atribut event seperti onerror maupun onload, sebagaimana dicontohkan pada cuplikan kode berikut. Pada saat pengguna lain membuka halaman yang menampilkan komentar tersebut, kode yang disisipkan akan tereksekusi secara otomatis pada peramban milik pengguna tersebut tanpa disadari, sehingga data sesi (cookie) milik pengguna turut terkirim ke server milik pelaku.

```

```

Dampak

Hal ini berimplikasi pada risiko pengambilalihan sesi (session hijacking), di mana pelaku dapat menggunakan cookie yang diperoleh untuk masuk ke akun korban tanpa memerlukan kredensial (nama pengguna dan kata sandi). Selain risiko tersebut, skrip yang disisipkan juga berpotensi digunakan untuk mengarahkan pengguna ke situs phishing, mencuri data formulir, maupun menyebarkan skrip serupa ke pengguna lain secara berantai.

Mitigasi

Guna mencegah kerentanan tersebut, terdapat beberapa langkah yang perlu dilakukan, yaitu: (1) melakukan penyaringan (sanitasi) dan validasi terhadap seluruh input pengguna sebelum disimpan maupun ditampilkan; (2) menggunakan textContent atau metode escaping karakter khusus HTML pada saat menampilkan data yang bersumber dari input pengguna, sebagaimana diuraikan pada Soal 1; serta (3) menerapkan kebijakan Content Security Policy (CSP) pada tingkat server guna membatasi sumber skrip yang diizinkan untuk dieksekusi pada halaman.

Soal 3 — Live Search dengan createElement & textContent

Implementasi berikut menyediakan fitur pencarian data secara langsung (real-time) menggunakan array data statis sebagai sumber. Proses penyaringan dilakukan melalui metode `array.filter()` setiap kali peristiwa (event) input terjadi pada kolom pencarian, kemudian hasil penyaringan ditampilkan kembali (re-render) ke dalam elemen daftar melalui kombinasi `document.createElement()` dan `textContent`, tanpa menggunakan `innerHTML`, guna memastikan keamanan terhadap potensi celah Cross-Site Scripting sebagaimana diuraikan pada Soal 2.

Berkas: 25120300030_Dharma.html

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Sistem Pencarian Data Tenant - Gaya JIEP</title>
<style>
  body { font-family: Georgia, 'Times New Roman', serif; max-width: 640px; margin: 40px auto; color: #1a1a1a; }
  h1 { font-size: 20px; border-bottom: 2px solid #14532d; padding-bottom: 8px; }
  #kolomPencarian { width: 100%; padding: 8px; font-size: 15px; box-sizing: border-box; margin-bottom: 16px; }
  #daftarHasil { list-style: none; padding: 0; }
  #daftarHasil li { padding: 8px 12px; border-bottom: 1px solid #ddd; }
  #keteranganJumlah { color: #555; font-size: 13px; margin-bottom: 8px; }
</style>
</head>
<body>

<h1>Sistem Pencarian Data Tenant Kawasan Industri</h1>
<p>Fitur berikut disediakan guna mempermudah proses pencarian data tenant secara langsung (<em>real-time</em>) berdasarkan kata kunci yang dimasukkan pengguna.</p>

<input type="text" id="kolomPencarian" placeholder="Masukkan kata kunci pencarian...">
<p id="keteranganJumlah"></p>
<ul id="daftarHasil"></ul>

<script>
  // Data sumber utama. Dalam penerapan produksi, data ini diperoleh dari basis data tenant.
  const dataSumberTenant = [
    "PT Sumber Makmur Sejahtera",
    "PT Cipta Karya Industri",
    "PT Nusantara Logistik Terpadu",
    "PT Anugerah Pratama Mandiri",
    "PT Karya Utama Perkasa",
    "PT Bumi Sentosa Abadi",
    "PT Graha Mitra Industri",
    "PT Sinar Cahaya Teknik",
    "PT Wahana Sukses Bersama",
    "PT Delta Prima Manufaktur"
  ];

  const elemenInput = document.querySelector("#kolomPencarian");
  const elemenDaftarHasil = document.querySelector("#daftarHasil");
  const elemenKeterangan = document.querySelector("#keteranganJumlah");

  // Fungsi ini bertugas mengosongkan daftar hasil sebelum dilakukan pengisian ulang.
  // Penghapusan dilakukan melalui removeChild, bukan melalui innerHTML, sesuai ketentuan.
  function kosongkanDaftarHasil() {
    while (elemenDaftarHasil.firstChild) {
      elemenDaftarHasil.removeChild(elemenDaftarHasil.firstChild);
    }
  }

  // Fungsi ini bertugas menyusun ulang (re-render) daftar hasil berdasarkan data yang diterima.
```

```

// Setiap baris dibentuk melalui createElement dan textContent, guna mencegah risiko
// Cross-Site Scripting (XSS) sebagaimana dijelaskan pada Soal 2.
function tampilkanDaftarHasil(dataYangDitampilkan) {
    kosongkanDaftarHasil();
    dataYangDitampilkan.forEach((namaTenant) => {
        const elemenBaris = document.createElement("li");
        elemenBaris.textContent = namaTenant;
        elemenDaftarHasil.appendChild(elemenBaris);
    });
    elemenKeterangan.textContent = "Ditemukan " + dataYangDitampilkan.length + " dari " +
dataSumberTenant.length + " data.";
}

// Proses penyaringan dilakukan setiap kali pengguna mengetik pada kolom pencarian
(event "input"),
// sehingga hasil dapat diperbarui secara langsung tanpa memerlukan aksi tambahan dari
pengguna.
elemenInput.addEventListener("input", (peristiwa) => {
    const kataKunci = peristiwa.target.value.trim().toLowerCase();
    const hasilPenyaringan = dataSumberTenant.filter((namaTenant) => {
        return namaTenant.toLowerCase().includes(kataKunci);
    });
    tampilkanDaftarHasil(hasilPenyaringan);
});

// Tampilan awal menampilkan seluruh data sebelum pengguna melakukan pencarian.
tampilkanDaftarHasil(dataSumberTenant);
</script>

</body>
</html>

```